Internal

TRAVAUX PRATIQUES HADOOP AVANCÉS

HDFS, administration, MapReduce, qualité et optimisation

Organisation générale

Cette série complète le cours Hadoop par des missions plus ouvertes. Les étudiants doivent proposer une démarche, exécuter les commandes, interpréter les sorties et justifier leurs choix.

TP	Thème	Compétence principale
TP 1	Architecte HDFS	Blocs, réplication, pannes et capacité
TP 2	Administrateur HDFS	Arborescence, commandes et métadonnées
TP 3	MapReduce qualité	Mapper/Reducer, compteurs et rejets
TP 4	Performance et petits fichiers	Formats, partitionnement et diagnostic
TP 5	Hadoop Investigation Challenge	Diagnostic d'incidents et reprise
TP 6	Écosystème Hadoop	Choix des outils et conception d'un pipeline Big Data

Internal

TP 1 - Architecte HDFS : capacité, réplication et panne

Contexte : une entreprise souhaite déposer dans HDFS un fichier de logs de 1 200 Mo sur un cluster de six DataNodes. La taille de bloc est de 128 Mo et le facteur de réplication est fixé à 3.

Objectifs

- Calculer le nombre de blocs logiques et physiques.
- Proposer une répartition cohérente des répliques.
- Simuler une panne et expliquer la re-réplication.
- Évaluer le coût de la tolérance aux pannes.

Mission A - Dimensionnement

1. Calculer le nombre de blocs logiques nécessaires.
2. Calculer la taille du dernier bloc.
3. Calculer le nombre de copies physiques.
4. Calculer le volume physique total occupé par les données, hors métadonnées.

Mission B - Placement

Construisez un tableau indiquant, pour chaque bloc B1 à B10, les trois DataNodes qui l'hébergent. Votre proposition doit respecter les contraintes suivantes :

- Jamais deux copies d'un même bloc sur le même DataNode.
- Répartition globalement équilibrée entre DN1 et DN6.
- Chaque bloc doit rester disponible après la perte d'un DataNode.

Mission C - Incident

5. Simuler la perte de DN3.
6. Identifier tous les blocs sous-répliqués.
7. Indiquer les copies encore disponibles.
8. Proposer les nouvelles copies à créer sur les DataNodes actifs.
9. Expliquer le rôle du NameNode, des heartbeats et des transferts entre DataNodes.

Bonus

- Comparer le stockage requis avec une réplication de 2.
- Expliquer le compromis entre coût de stockage et disponibilité.
- Proposer le comportement attendu si DN3 et DN4 deviennent indisponibles simultanément.

Livrable

- Une page de calculs.
- Un tableau de placement initial.
- Un tableau après panne et re-réplication.
- Une conclusion de 10 lignes maximum.

Mise en pratique adaptée à Google Colab

Google Colab permet de réaliser le découpage en blocs, les commandes HDFS et l'analyse avec fsck en mode pseudo-distribué mono-nœud. Il ne permet pas de démontrer fidèlement six machines persistantes : la réplication physique et la tolérance aux pannes seront donc traitées dans un exercice multi-nœuds séparé.

Checklist Colab avant de commencer les TP 1 et 5

Exécuter cette checklist au début de la séance. Ne commencer le TP que lorsque tous les contrôles obligatoires sont validés.

1 - Vérifier la session Colab

- Le notebook est ouvert dans une session active.
- L'espace disque disponible est suffisant pour Hadoop et les fichiers de test.
- La session n'a pas été réinitialisée depuis l'installation de Hadoop.

```
# Affiche l'espace disque disponible dans /content
!df -h /content

# Affiche la mémoire disponible
!free -h
```

Résultat attendu : plusieurs gigaoctets doivent rester disponibles dans */content*. Si la session a été réinitialisée, reprendre l'installation depuis la cellule 0.

2 - Vérifier Java et Hadoop

```
# Vérifie la version de Java
!java -version

# Vérifie que Hadoop est accessible
!hadoop version
```

Résultat attendu : Java 17 et Hadoop 3.5.0 sont affichés. Si *hadoop: command not found* apparaît, rejouer la cellule d'initialisation des variables puis, si nécessaire, les cellules d'installation.

3 - Vérifier les variables d'environnement

```
# Affiche les chemins utilisés par Hadoop
import os
print("HADOOP_HOME =", os.environ.get("HADOOP_HOME"))
print("JAVA_HOME   =", os.environ.get("JAVA_HOME"))
print("Hadoop dans PATH =", "/content/hadoop-3.5.0/bin" in os.environ.get("PATH", ""))
```

Résultat attendu : HADOOP_HOME vaut */content/hadoop-3.5.0*, JAVA_HOME vaut */usr/lib/jvm/java-17-openjdk-amd64* et le dernier contrôle affiche *True*.

4 - Vérifier les services HDFS

```
# Liste les processus Java Hadoop
!jps

# Contrôle les DataNodes enregistrés
!hdfs dfsadmin -report | grep -E "Live datanodes|Dead datanodes"
```

Internal


```
# Vérifie le safe mode
!hdfs dfsadmin -safemode get
```

Résultat attendu : *jps* affiche NameNode, DataNode et SecondaryNameNode ; le rapport indique *Live datanodes (1)* et *Dead datanodes (0)* ; le safe mode est désactivé. Si les services sont absents, rejouer la cellule de démarrage HDFS. Ne pas reformater le NameNode pendant le TP 5, car cela supprimerait l'état utile au diagnostic.

5 - Vérifier la configuration HDFS

```
# Affiche la taille de bloc en octets
!hdfs getconf -confKey dfs.blocksize

# Affiche la réplication par défaut
!hdfs getconf -confKey dfs.replication
```

Résultat attendu dans Colab : *134217728* pour la taille de bloc, soit 128 Mo, et *1* pour la réplication par défaut. Le TP 1 demandera ponctuellement une réplication de 3 afin d'observer la sous-réplication sur un cluster mono-nœud.

6 - Vérifier l'écriture et la lecture dans HDFS

```
# Crée un répertoire et un fichier test dans HDFS
!hdfs dfs -mkdir -p /check_colab
!echo "HDFS_OK" | hdfs dfs -put -f - /check_colab/test.txt

# Relit le fichier
!hdfs dfs -cat /check_colab/test.txt

# Supprime le test
!hdfs dfs -rm -r -f /check_colab
```

Résultat attendu : la cellule affiche *HDFS_OK*, puis confirme la suppression du répertoire. Ce contrôle prouve que le NameNode et le DataNode permettent l'écriture et la lecture.

7 - Vérifier YARN pour le TP 5

```
# Vérifie les processus ResourceManager et NodeManager
!jps

# Liste les nœuds de calcul YARN
!yarn node -list
```

Résultat attendu : ResourceManager et NodeManager apparaissent dans *jps* et YARN indique *Total Nodes:1* avec un nœud dans l'état *RUNNING*. Si YARN n'est pas démarré, configurer *mapred-site.xml* et *yarn-site.xml*, puis exécuter *start-yarn.sh*. YARN est nécessaire pour les incidents du TP 5 liés aux applications MapReduce et aux journaux d'exécution.

8 - Préparer les preuves et les fichiers

- Créer un dossier local */content/preuves_tp* pour conserver les sorties utiles.
- Vérifier que les fichiers et scripts des TP précédents sont présents avant le TP 5.
- Conserver une copie des commandes exécutées, des messages d'erreur et des corrections.

```
# Prépare le dossier local de preuves
!mkdir -p /content/preuves_tp

# Affiche son contenu
!ls -la /content/preuves_tp
```

Validation finale

Contrôle	TP 1	TP 5
Java et Hadoop accessibles	Obligatoire	Obligatoire
NameNode et DataNode actifs	Obligatoire	Obligatoire
Écriture et lecture HDFS validées	Obligatoire	Obligatoire
ResourceManager et NodeManager actifs	Non requis	Requis pour les incidents YARN et MapReduce
Fichiers et scripts des TP précédents	Non requis	Requis selon l'incident

Décision : commencer le TP 1 lorsque Java, Hadoop et HDFS sont validés. Commencer le TP 5 lorsque ces contrôles sont validés et que YARN ainsi que les fichiers nécessaires aux incidents sont disponibles. Dans Colab, ne jamais arrêter le seul DataNode dans le cadre du TP 1.

Partie A - TP exécutable dans Google Colab

Ce qui sera démontré

- Installation temporaire de Java, SSH et Hadoop dans la machine virtuelle Colab.
- Démarrage d'un NameNode et d'un DataNode sur la même machine.
- Création d'un fichier de 300 Mo pour limiter le temps et l'espace utilisés.
- Découpage en trois blocs : 128 Mo, 128 Mo et 44 Mo.
- Configuration d'une réplication demandée de 3 et constat d'une seule copie physique disponible.
- Observation du statut de sous-réplication avec fsck.

Mode d'emploi dans Google Colab

1. Créer un nouveau notebook Colab.
2. Copier chaque bloc ci-dessous dans **une cellule de code distincte**.
3. Exécuter les cellules dans l'ordre, de la cellule 0 à la cellule 10.
4. Ne pas utiliser « Réinitialiser l'environnement d'exécution » pendant le TP : cette action supprime Hadoop et les données temporaires.
5. Si la session Colab se déconnecte ou redémarre, reprendre depuis la cellule 0.

Cellule 0 - Initialiser les variables Colab

```
# Importe le module qui permet de gérer les variables d'environnement
import os

# Version de Hadoop utilisée dans ce TP
HADOOP_VERSION = "3.5.0"

# Construit le chemin d'installation : /content/hadoop-3.5.0
HADOOP_HOME = f"/content/hadoop-{HADOOP_VERSION}"

# Indique le dossier dans lequel Java 17 est installé
JAVA_HOME = "/usr/lib/jvm/java-17-openjdk-amd64"

# Rend le chemin de Hadoop accessible aux programmes lancés dans
```

```
Colab
os.environ["HADOOP_HOME"] = HADOOP_HOME

# Indique à Hadoop quelle installation Java utiliser
os.environ["JAVA_HOME"] = JAVA_HOME

# Ajoute les commandes Hadoop et les scripts d'administration au PATH
# L'ancien PATH est conservé pour garder les autres commandes du
système
os.environ["PATH"] = f"{HADOOP_HOME}/bin:{HADOOP_HOME}/sbin:" +
os.environ["PATH"]

# Affiche les deux chemins afin de vérifier la configuration
print("HADOOP_HOME =", os.environ["HADOOP_HOME"])
print("JAVA_HOME   =", os.environ["JAVA_HOME"])
```

Résultat attendu : les deux chemins sont affichés. Cette cellule centralise les variables utilisées pendant tout le notebook. Elle doit être rejouée après tout redémarrage de la session.

Cellule 1 - Installer les composants

```
!apt-get update -qq
!apt-get install -y -qq openjdk-17-jdk-headless openssh-server rsync wget
!java -version
```

À quoi servent-ils ? Java exécute Hadoop ; OpenSSH permet aux scripts Hadoop de lancer les démons ; rsync est utilisé par certains scripts d'administration ; wget télécharge l'archive Hadoop.

Résultat attendu : la dernière commande affiche une version OpenJDK 17. Les téléchargements peuvent prendre quelques minutes.

Cellule 2 - Télécharger Hadoop 3.5.0

```
!wget -q https://downloads.apache.org/hadoop/common/hadoop-3.5.0/hadoop-3.5.0.tar.gz
!tar -xzf hadoop-3.5.0.tar.gz -C /content
%env JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
%env HADOOP_HOME=/content/hadoop-3.5.0
%env PATH=/content/hadoop-3.5.0/bin:/content/hadoop-3.5.0/sbin:$PATH
!hadoop version
```

Résultat attendu : Hadoop 3.5.0 et les informations de compilation s'affichent. Si le miroir principal est temporairement indisponible, l'enseignant fournit l'archive ou un miroir Apache validé.

Cellule 3 - Configurer le mode pseudo-distribué

```
%%bash
export HADOOP_HOME=/content/hadoop-3.5.0
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
sed -i 's|^export JAVA_HOME=.*|export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64|'
$HADOOP_HOME/etc/hadoop/hadoop-env.sh
cat > $HADOOP_HOME/etc/hadoop/core-site.xml <<'EOF'
<configuration>
```

Internal


```
  <property><name>fs.defaultFS</name><value>hdfs://localhost:9000</value></property>
</configuration>
EOF
cat > $HADOOP_HOME/etc/hadoop/hdfs-site.xml <<'EOF'
<configuration>
  <property><name>dfs.replication</name><value>1</value></property>
  <property><name>dfs.blocksize</name><value>134217728</value></property>

<property><name>dfs.namenode.name.dir</name><value>file:///content/hdfs/name</value></property
>
  <property><name>dfs.datanode.data.dir</name><value>file:///content/hdfs/data</value></property>
</configuration>
EOF
```

Résultat attendu : aucune sortie particulière. Les deux fichiers XML doivent exister et la réplication par défaut est volontairement fixée à 1, car Colab ne possède qu'un DataNode réel.

Cellule 4 - Préparer SSH et démarrer HDFS

```
%%bash
export HADOOP_HOME=/content/hadoop-3.5.0
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
service ssh start
mkdir -p ~/.ssh
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa >/dev/null 2>&1 || true
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh-keyscan -H localhost >> ~/.ssh/known_hosts 2>/dev/null
rm -rf /content/hdfs
hdfs namenode -format -force -nonInteractive
$HADOOP_HOME/sbin/start-dfs.sh
jps
```

Résultat attendu : la fin de la cellule affiche notamment NameNode, DataNode, SecondaryNameNode et Jps. Le formatage ne doit être exécuté qu'à l'initialisation de cette machine Colab temporaire.

Cellule 5 - Vérifier le cluster Colab

```
!hdfs dfsadmin -report
!hdfs getconf -confKey dfs.blocksize
!hdfs getconf -confKey dfs.replication
```

Résultat attendu : Live datanodes (1), taille de bloc 134217728 octets et réplication par défaut 1.

Cellule 5 bis - Diagnostic rapide en cas de problème

```
%%bash
export HADOOP_HOME=/content/hadoop-3.5.0
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$HADOOP_HOME/bin:$HADOOP_HOME/sbin:$PATH
```

Internal


```
echo "=== Versions ==="
java -version 2>&1 | head -n 1
hadoop version | head -n 1

echo "=== Processus Java ==="
jps

echo "=== Safe mode ==="
hdfs dfsadmin -safemode get

echo "=== DataNodes ==="
hdfs dfsadmin -report | grep -E "Live datanodes|Dead datanodes"

echo "=== Configuration ==="
echo -n "Taille de bloc : "
hdfs getconf -confKey dfs.blocksize
echo -n "Réplication par défaut : "
hdfs getconf -confKey dfs.replication
```

Résultat attendu : Java 17, Hadoop 3.5.0, les processus NameNode et DataNode, un safe mode désactivé, un DataNode vivant, une taille de bloc de 134217728 octets et une réplication par défaut égale à 1. Si NameNode ou DataNode n'apparaît pas, rejouer la cellule 4 avant de poursuivre.

Cellule 6 - Créer le fichier pratique de 300 Mo

```
!mkdir -p /content/tp1
!dd if=/dev/zero of=/content/tp1/logs_300Mo.bin bs=1M count=300 status=progress
!ls -lh /content/tp1/logs_300Mo.bin
```

Résultat attendu : un fichier d'environ 300 Mo, soit 314 572 800 octets. Le calcul théorique est 300 ÷ 128 = 2,34375 : il faut donc trois blocs, dont le dernier mesure 44 Mo.

Cellule 7 - Charger le fichier avec une réplication demandée de 3

```
!hdfs dfs -mkdir -p /tp1/input
!hdfs dfs -rm -f /tp1/input/logs_300Mo.bin
!hdfs dfs -Ddfs.blocksize=134217728 -Ddfs.replication=3 -put /content/tp1/logs_300Mo.bin /tp1/input/
!hdfs dfs -ls -h /tp1/input
```

Résultat attendu : le fichier apparaît avec une taille de 300 Mo et une réplication demandée de 3. Cependant, une seule copie physique peut exister, car le cluster ne contient qu'un DataNode.

Cellule 8 - Observer les blocs et la sous-réplication

```
!hdfs dfs -stat "Nom=%n Taille=%b Réplication=%r" /tp1/input/logs_300Mo.bin
!hdfs fsck /tp1/input/logs_300Mo.bin -files -blocks -locations
```

Résultat attendu : trois blocs logiques. Les deux premiers ont une longueur de 134 217 728 octets et le dernier 46 137 344 octets, soit 44 Mo. Chaque ligne affiche généralement Live_repl=1, même si la réplication enregistrée est 3. Le fichier est lisible, mais HDFS signale une sous-réplication car il ne peut pas placer les copies 2 et 3 sur des nœuds différents.

Internal

Cellule 9 - Vérifier la lecture et l'intégrité

```
!md5sum /content/tp1/logs_300Mo.bin
!hdfs dfs -cat /tp1/input/logs_300Mo.bin | md5sum
```

Résultat attendu : les deux empreintes sont identiques. Le fichier lu depuis HDFS est identique au fichier local.

Cellule 10 - Nettoyer

```
!hdfs dfs -rm -r -f /tp1
!rm -rf /content/tp1
```

Attention : la machine virtuelle Colab et son stockage local sont temporaires. Après déconnexion ou réinitialisation, l'installation Hadoop et les données peuvent disparaître ; il faut alors rejouer les cellules depuis le début.

Travail demandé dans Colab

1. Expliquer pourquoi le fichier de 300 Mo produit trois blocs.
2. Reporter les longueurs exactes affichées par fsck.
3. Expliquer la différence entre réplication demandée et copies réellement disponibles.
4. Justifier pourquoi il ne faut pas arrêter le seul DataNode de Colab.
5. Comparer les résultats pratiques de 300 Mo avec le calcul théorique du fichier de 1 200 Mo.

Partie B - Exercice optionnel : démontrer réellement six DataNodes

Cet exercice ne doit pas être exécuté dans Google Colab. Il nécessite un poste suffisamment puissant ou six machines virtuelles reliées sur un réseau privé. La solution la plus accessible pour un cours est Docker Compose : un conteneur NameNode et six conteneurs DataNode.

Liste des éléments à installer

- Un système 64 bits : Windows 11 avec WSL 2, Linux Ubuntu ou macOS.
- Docker Desktop sur Windows/macOS, ou Docker Engine sur Linux.
- Docker Compose, inclus dans Docker Desktop ou disponible comme module sur Linux.
- Git pour récupérer le dossier pédagogique préparé par l'enseignant.
- Au moins 8 Go de mémoire vive disponible ; 12 à 16 Go sont recommandés pour une exécution confortable.
- Au moins 15 Go d'espace disque libre.
- Un terminal Bash ou PowerShell.
- Facultatif : Visual Studio Code et son extension Docker pour examiner les conteneurs et journaux.

Alternative avec de vraies machines virtuelles : installer sur les sept machines Linux Java 17, Hadoop 3.5.0, OpenSSH Server, rsync et des noms d'hôtes résolus par DNS ou par le fichier hosts. Une machine joue le rôle de NameNode et six machines jouent le rôle de DataNode. Les scripts Hadoop nécessitent SSH pour administrer les démons distants.

Architecture de l'exercice Docker

Service	Rôle	Volume persistant
namenode	Métadonnées HDFS et interface d'administration	namenode_data
datanode1 à datanode6	Stockage des blocs HDFS	dn1_data à dn6_data
Réseau hadoop-net	Communication privée et résolution des noms de services	Non applicable

Préparation fournie par l'enseignant

L'enseignant prépare un dossier *hadoop-6dn* comprenant un fichier *compose.yaml*, une image Hadoop 3.5.0 validée, les fichiers *core-site.xml* et *hdfs-site.xml*, ainsi qu'un script de démarrage. L'image doit utiliser Java 17. Le fichier HDFS doit définir une

Internal

taille de bloc de 128 Mo et une réplication de 3. Le réseau ne doit pas être exposé publiquement ; seul le port de l'interface NameNode peut être publié sur la machine locale.

Étape 1 - Vérifier les installations

```
docker --version
docker compose version
git --version
```

Résultat attendu : les trois commandes affichent une version. Si *docker compose* est inconnu sous Linux, installer le module Docker Compose avant de continuer.

Étape 2 - Démarrer le cluster

```
git clone <URL_DU_DEPOT_FOURNI_PAR_L_ENSEIGNANT> hadoop-6dn
cd hadoop-6dn
docker compose up -d
docker compose ps
```

Résultat attendu : sept services sont en état *running* ou *healthy* : namenode, datanode1, datanode2, datanode3, datanode4, datanode5 et datanode6.

Étape 3 - Contrôler HDFS

```
docker compose exec namenode hdfs dfsadmin -report
```

Résultat attendu : Live datanodes (6) et Dead datanodes (0). Relever les six noms ou adresses affichés.

Étape 4 - Créer et charger le fichier

```
docker compose exec namenode bash -lc "dd if=/dev/zero of=/tmp/logs_1200Mo.bin bs=1M
count=1200 status=progress"
docker compose exec namenode hdfs dfs -mkdir -p /tp1/input
docker compose exec namenode hdfs dfs -Ddfs.blocksize=134217728 -Ddfs.replication=3 -put
/tmp/logs_1200Mo.bin /tp1/input/
```

Résultat attendu : le chargement réussit. Le fichier est découpé en dix blocs logiques et chaque bloc possède progressivement trois copies.

Étape 5 - Relever le placement initial

```
docker compose exec namenode hdfs fsck /tp1/input/logs_1200Mo.bin -files -blocks -locations
```

Résultat attendu : neuf blocs de 128 Mo, un bloc final de 48 Mo et Live_repl=3 pour chaque bloc. Les étudiants construisent le tableau réel Bloc, Taille, Copie 1, Copie 2 et Copie 3.

Étape 6 - Provoquer la panne de DN3

```
docker compose stop datanode3
docker compose ps
docker compose exec namenode hdfs dfsadmin -report
```

Internal

Résultat attendu : datanode3 est arrêté. La détection HDFS n'est pas toujours instantanée : il faut attendre le délai de heartbeat. Le rapport finit par indiquer cinq DataNodes vivants et un DataNode mort.

Étape 7 - Vérifier la disponibilité et la re-réplication

```
docker compose exec namenode hdfs dfs -cat /tp1/input/logs_1200Mo.bin > /dev/null
echo $?
docker compose exec namenode hdfs fsck /tp1/input/logs_1200Mo.bin -files -blocks -locations
```

Résultat attendu : le code de lecture est 0. Les blocs auparavant présents sur DN3 peuvent afficher temporairement Live_repl=2, puis revenir à Live_repl=3 lorsque le NameNode demande de nouvelles copies sur les cinq DataNodes actifs. Les blocs restent disponibles pendant la panne.

Étape 8 - Redémarrer DN3

```
docker compose start datanode3
docker compose exec namenode hdfs dfsadmin -report
docker compose exec namenode hdfs fsck /tp1/input/logs_1200Mo.bin -files -blocks -locations
```

Résultat attendu : le rapport revient à six DataNodes vivants. DN3 ne récupère pas nécessairement les mêmes blocs, car HDFS peut avoir rétabli la réplication sur d'autres nœuds.

Étape 9 - Tester une deuxième panne sans casser le cluster

Après retour complet à Live_repl=3, arrêter un autre DataNode et répéter les contrôles. Ne pas arrêter plusieurs nœuds simultanément avant d'avoir vérifié le placement réel. Avec une réplication de 3, la perte d'un nœud est tolérée ; la tolérance à deux pannes dépend du fait que les trois copies d'un bloc ne soient pas toutes concernées.

Étape 10 - Nettoyer

```
docker compose exec namenode hdfs dfs -rm -r -f /tp1
docker compose down
docker compose down -v
```

Précaution : la dernière commande supprime aussi les volumes et toutes les données HDFS du laboratoire. Elle ne doit être exécutée qu'après sauvegarde des preuves demandées.

Questions de l'exercice six DataNodes

1. Quels blocs étaient hébergés sur DN3 avant la panne ?
2. Combien de copies restaient immédiatement après la détection de la panne ?
3. Sur quels DataNodes les nouvelles copies ont-elles été créées ?
4. Le fichier est-il resté lisible pendant toute l'expérience ? Justifier avec une commande et son code retour.
5. Comparer le placement réel au tableau théorique du TP.
6. Expliquer le rôle du NameNode, des heartbeats et des transferts directs entre DataNodes.

Livrables

- Capture de *docker compose ps* montrant les sept services.
- Rapport HDFS avec six DataNodes avant la panne.
- Tableau de placement des dix blocs avant la panne.
- Rapport HDFS avec cinq DataNodes vivants et un mort.
- Deux sorties fsck : avant la panne et après re-réplication.
- Preuve de lecture réussie pendant la panne.
- Conclusion expliquant la différence entre Colab mono-nœud et le cluster réellement distribué.

Internal

Éléments à ajouter au compte rendu

- Les calculs théoriques du fichier de 1 200 Mo.
- Les sorties Colab montrant trois blocs pour le fichier de 300 Mo.
- L'explication de la sous-réplication dans Colab.
- Pour l'option multi-nœuds, le tableau réel de placement et l'évolution après la perte de DN3.
- Une conclusion distinguant simulation, pseudo-distribution et distribution réelle.

TP 2 - Administrateur HDFS : organiser et contrôler un espace de données

Contexte : vous administrez l'espace HDFS d'une entreprise qui reçoit des ventes, des clients, des produits et des logs web. Vous devez créer une organisation raw/curated/analytics, charger les fichiers et contrôler leur stockage.

Jeux de données à préparer

Fichier	Colonnes minimales	Anomalies à inclure
ventes.csv	date, pays, produit, quantité, prix	1 ligne en double
clients.csv	client_id, pays, segment	1 pays manquant
produits.csv	produit_id, famille, prix_ref	1 prix invalide
logs_web.csv	timestamp, ip, page, statut	Plusieurs statuts 404

Mission A - Préparer les services

```
jps
hdfs dfsadmin -report
yarn node -list
```

- Vérifier la présence du NameNode et du DataNode.
- Relever le nombre de DataNodes vivants.
- Expliquer la différence entre le rôle de HDFS et celui de YARN.

Mission B - Construire l'arborescence

```
/data/
  raw/
    sales/ingestion_date=2026-10-11/
    customers/ingestion_date=2026-10-11/
    products/ingestion_date=2026-10-11/
    web_logs/ingestion_date=2026-10-11/
  curated/
    sales/year=2026/month=10/
  analytics/
  rejects/
```

- Créer tous les répertoires avec hdfs dfs -mkdir -p.
- Justifier les zones raw, curated, analytics et rejects.
- Expliquer le rôle de ingestion_date dans la zone raw.

Mission C - Charger et inspecter

```
hdfs dfs -put -f <fichier_local> <chemin_hdfs>
hdfs dfs -ls -R /data
```

Internal


```
hdfs dfs -du -h /data/raw
hdfs dfs -cat <chemin_fichier> | head
hdfs fsck <chemin_fichier> -files -blocks -locations
```

10. Charger les quatre fichiers.
11. Vérifier leur présence et lire un extrait.
12. Observer leur taille logique.
13. Identifier le nombre de blocs et leur réplication.
14. Comparer le chemin local et le chemin HDFS.

Mission D - Droits et cycle de vie

```
hdfs dfs -chmod 750 /data/curated
hdfs dfs -chmod 770 /data/analytics
hdfs dfs -touchz /data/analytics/_READY
hdfs dfs -stat "%n %b %r" <chemin_fichier>
```

- Interpréter les droits 750 et 770.
- Expliquer l'intérêt d'un fichier marqueur _READY.
- Proposer une règle simple d'archivage ou de suppression des données raw.

Contrôles attendus

- Capture de jps et hdfs dfsadmin -report.
- Arborescence complète /data.
- Sortie fsck d'au moins deux fichiers.
- Tableau comparatif local/HDFS.
- Explication des zones et des droits.

Internal

TP 3 - MapReduce avancé : chiffre d'affaires et qualité des données

Mission : développer un traitement Hadoop Streaming qui calcule le chiffre d'affaires par pays tout en détectant les lignes invalides grâce à des compteurs métier.

Données d'entrée

```
date;pays;produit;quantite;prix_unitaire
2026-10-01;FR;P01;2;10.00
2026-10-01;BE;P02;1;15.00
2026-10-02;FR;P03;3;12.00
2026-10-02;BE;P01;4;10.00
2026-10-03;ES;P02;2;15.00
2026-10-03;FR;P01;1;10.00
2026-10-04;DE;P04;abc;20.00
2026-10-04;;P05;2;9.50
2026-10-05;FR;P06;-1;30.00
```

Règles de qualité

- Le pays est obligatoire.
- La quantité doit être un entier strictement positif.
- Le prix doit être numérique et strictement positif.
- L'en-tête ne doit produire aucune sortie.

Mission A - Mapper

15. Lire chaque ligne depuis l'entrée standard.
16. Contrôler le format et les règles de qualité.
17. Émettre pays et montant pour les lignes valides.
18. Émettre un compteur différent pour pays manquant, quantité invalide et valeur négative.

Format de compteur Hadoop Streaming :

```
reporter:counter:Qualite,Quantite_invalide,1
```

Mission B - Reducer

- Additionner les montants par pays.
- Produire un résultat arrondi à deux décimales.
- Ne pas coder en dur la liste des pays.

Mission C - Test local

```
cat ventes.csv | ./mapper.py 1> sortie_map.txt 2> erreurs_map.txt
cat ventes.csv | ./mapper.py | sort | ./reducer.py
```

- Comparer la sortie du Mapper et le résultat final.
- Expliquer pourquoi sort simule le Shuffle/Sort.
- Vérifier manuellement les totaux.

Mission D - Exécution Hadoop

```
STREAMING_JAR=$(find "$HADOOP_HOME/share/hadoop/tools/lib" -name 'hadoop-streaming-*.jar' | head -1)
```

Internal


```
hdfs dfs -rm -r -f /tp_avance/ca/output
hadoop jar "$STREAMING_JAR" -D mapreduce.job.name=TP_Avance_CA_Qualite -D mapreduce.job.reduces=1 -files mapper.py,reducer.py -mapper "python3 mapper.py" -reducer "python3 reducer.py" -input /tp_avance/ca/input -output /tp_avance/ca/output 2>&1 | tee execution.log
```

Mission E - Analyse

19. Relever Map input records, Map output records et Reduce output records.
20. Relever les compteurs métier.
21. Expliquer la différence entre lignes lues, lignes valides, lignes rejetées et groupes Reduce.
22. Provoquer une erreur de dossier output existant, puis la corriger.

TP 4 - Performance HDFS : petits fichiers, formats et partitionnement

Contexte : une application dépose un fichier CSV par minute. Après plusieurs mois, HDFS contient un grand nombre de petits fichiers. Les requêtes deviennent coûteuses et la charge du NameNode augmente.

Mission A - Reproduire le problème

```
mkdir -p ~/tp4-smallfiles/source
for i in $(seq -w 1 200); do
  echo "2026-10-11;FR;P01;$i" > ~/tp4-smallfiles/source/event_$i.csv
done
hdfs dfs -mkdir -p /tp4/raw/events
hdfs dfs -put ~/tp4-smallfiles/source/*.csv /tp4/raw/events/
hdfs dfs -count -q -h /tp4/raw/events
hdfs dfs -du -h /tp4/raw/events | head
```

23. Compter les fichiers.
24. Comparer le volume total aux nombres d'objets HDFS.
25. Expliquer pourquoi ce cas surcharge les métadonnées.

Mission B - Compacter

```
cat ~/tp4-smallfiles/source/*.csv > ~/tp4-smallfiles/events_compactes.csv
hdfs dfs -mkdir -p /tp4/curated/events
hdfs dfs -put -f ~/tp4-smallfiles/events_compactes.csv /tp4/curated/events/
hdfs fsck /tp4/curated/events/events_compactes.csv -files -blocks -locations
```

- Comparer le nombre de fichiers avant et après compactage.
- Comparer le nombre de blocs.
- Expliquer le gain pour le NameNode et pour les traitements.

Mission C - Concevoir un partitionnement

Proposer une arborescence pour des événements consultés principalement par date et pays. Exemple de forme attendue :

```
/data/curated/events/year=AAAA/month=MM/day=JJ/country=XX/
```

26. Proposer les clés de partitionnement.
27. Justifier l'ordre des partitions.
28. Expliquer le risque de créer des partitions trop fines.
29. Comparer CSV gzip, Avro, Parquet et ORC pour ce cas analytique.

Internal

Mission D - Décision argumentée

Rédiger une recommandation de 15 lignes maximum répondant aux questions suivantes :

- Faut-il conserver un fichier par minute ?
- Quand déclencher une compaction ?
- Quel format recommander pour l'analyse ?
- Quelle taille de fichier cible viser sans donner une règle universelle ?
- Quels contrôles automatiser ?

Internal

TP 5 - Hadoop Investigation Challenge

Organisation : chaque groupe reçoit un environnement présentant plusieurs anomalies. L'objectif n'est pas seulement de corriger, mais de démontrer la démarche de diagnostic.

Scénarios d'incident

Incident	Symptôme remis au groupe	Résultat attendu
A	hadoop: command not found	Vérifier variables et recharger .bashrc
B	DataNode absent dans jps	Consulter les journaux et corriger la cause
C	Safe mode actif	Diagnostiquer avant toute sortie du safe mode
D	FileAlreadyExistsException	Supprimer ou changer le chemin de sortie
E	Permission denied dans HDFS	Vérifier propriétaire, groupe et droits
F	Mapper non exécutable	Contrôler chmod et commande -mapper
G	Aucun fichier trouvé	Vérifier chemin local, chemin HDFS et glob
H	Résultat incorrect	Comparer entrée, compteurs, Mapper, tri et Reducer

Boîte à outils autorisée

```
jps
hdfs dfsadmin -report
hdfs dfs -ls -R /
hdfs dfs -test -e <chemin>
hdfs dfs -stat "%n %b %r" <chemin>
hdfs fsck <chemin> -files -blocks -locations
yarn application -list -appStates ALL
yarn logs -applicationId <application_id>
tail -n 50 $HADOOP_HOME/logs/<journal>
echo $HADOOP_HOME
echo $JAVA_HOME
ls -l <script>
```

Méthode obligatoire

30. Décrire précisément le symptôme.
31. Formuler au moins deux hypothèses.
32. Choisir une commande de vérification pour chaque hypothèse.
33. Identifier la cause avec une preuve.
34. Appliquer la correction minimale.
35. Vérifier que le service ou le job fonctionne.
36. Documenter une mesure préventive.

Fiche d'incident à compléter

Champ	Contenu attendu
Identifiant	Incident A à H
Symptôme	Message exact ou comportement observé
Hypothèses	Causes possibles avant investigation
Preuves	Commandes et extraits utiles
Cause racine	Cause confirmée
Correction	Action appliquée
Validation	Preuve de retour au fonctionnement
Prévention	Contrôle ou règle à ajouter

Restitution

- Présentation de 5 minutes par groupe.
- Une fiche par incident traité.
- Démonstration d'au moins une correction.
- Conclusion sur l'importance de l'observabilité et du rejeu.

TP 6 - Concevoir un pipeline avec l'écosystème Hadoop

Contexte : une entreprise reçoit chaque jour des ventes, des référentiels clients et produits, ainsi que des logs applicatifs. Les données doivent être collectées, stockées, interrogées et traitées. Votre groupe doit proposer une architecture cohérente en sélectionnant les composants adaptés de l'écosystème Hadoop.

Objectifs

- Identifier le rôle de Hive, HBase, Pig, Flume, ZooKeeper et Spark.
- Distinguer stockage, collecte, traitement, requêtage et coordination.
- Choisir un outil selon un besoin métier et justifier ce choix.
- Concevoir un pipeline complet autour de HDFS.
- Comparer les approches historiques avec les solutions aujourd'hui privilégiées.

Situation de départ

L'entreprise met à disposition les sources suivantes :

Source	Contenu	Besoin principal
ventes.csv	Transactions quotidiennes	Analyse du chiffre d'affaires
clients.csv et produits.csv	Référentiels métier	Jointures et enrichissement
logs_applicatifs.log	Événements générés en continu	Collecte vers HDFS
etat_commandes	Statut évolutif des commandes	Consultation rapide par identifiant

Mission A - Cartographier les composants

Compléter le tableau sans recopier le cours. Pour chaque composant, préciser son rôle, le type de données traité et un exemple d'utilisation dans le contexte proposé.

Composant	Catégorie à identifier	Question à traiter
Hive	Requêtage SQL	Pourquoi l'utiliser pour analyser les ventes ?
HBase	Base NoSQL en colonnes	Pourquoi l'utiliser pour consulter un statut par identifiant ?
Pig	Flux de transformation historique	Quel traitement de préparation pourrait être décrit ?
Flume	Collecte de logs	Comment acheminer les logs vers HDFS ?
ZooKeeper	Coordination distribuée	Quels services distribués doivent être coordonnés ?
Spark	Traitement distribué	Quels calculs ou nettoyages lui confier ?

Mission B - Choisir le bon outil

Pour chaque besoin, sélectionner un composant principal et justifier la réponse en trois lignes maximum.

37. Un analyste veut calculer le chiffre d'affaires mensuel par pays avec une syntaxe proche de SQL.
38. Une application doit retrouver immédiatement l'état d'une commande à partir de son identifiant.

Internal

39. Des fichiers de logs sont produits en continu sur plusieurs serveurs et doivent être centralisés dans HDFS.
40. Un traitement doit nettoyer, joindre et agréger plusieurs millions de lignes.
41. Plusieurs services distribués doivent partager un état de coordination et gérer le basculement.
42. Un ancien flux Pig doit être modernisé : proposer une cible et expliquer le gain attendu.

Mission C - Concevoir l'architecture

Construire un schéma contenant obligatoirement les éléments suivants : sources de données, collecte, HDFS, traitement, stockage spécialisé, interrogation et restitution. Les flèches doivent être nommées avec la nature du flux.

Contraintes à respecter :

- HDFS reste la zone de stockage distribuée centrale.
- Les logs sont collectés avant leur stockage.
- Les données analytiques peuvent être interrogées en SQL.
- Les statuts à accès rapide sont séparés des fichiers analytiques.
- Spark réalise au moins une étape de nettoyage ou d'agrégation.
- ZooKeeper ne doit pas être représenté comme un moteur de stockage ou de calcul.

Mission D - Décrire le pipeline de traitement

Rédiger le déroulement du pipeline sous la forme de 8 à 12 étapes numérotées. Chaque étape doit préciser : l'entrée, le composant utilisé, l'action effectuée et la sortie produite.

Exemple de structure attendue : « Les logs applicatifs sont collectés par Flume, puis déposés dans /data/raw/logs/ sur HDFS. »

Mission E - Mini-cas Hive et Spark

Sans imposer l'installation complète de tous les composants, produire les éléments suivants :

- Une instruction HiveQL de création d'une table externe sur les ventes stockées dans HDFS.
- Une requête HiveQL calculant le chiffre d'affaires par pays.
- Un pseudo-code Spark qui lit les ventes, élimine les quantités négatives et agrège le montant par pays.
- Une comparaison courte entre Hive et Spark : objectif, type de traitement et utilisateur cible.

Mission F - Prendre du recul

Rédiger une recommandation de 15 lignes maximum répondant aux questions suivantes :

- Quels composants sont indispensables dans l'architecture proposée ?
- Quels composants sont historiques ou à éviter dans un nouveau projet ?
- Dans quels cas HBase apporte-t-il une valeur que Hive n'apporte pas ?
- Pourquoi Spark ne remplace-t-il pas HDFS ?
- Quels risques apparaissent si chaque outil est ajouté sans besoin clairement identifié ?

Livrables

- Le tableau de cartographie complété.
- Les réponses argumentées aux six besoins.
- Un schéma d'architecture lisible et légendé.
- La description numérotée du pipeline.
- Les propositions HiveQL et le pseudo-code Spark.
- La recommandation finale.

Modalité de travail et évaluation

Critère	Points
Compréhension du rôle des composants	6
Pertinence des choix techniques	5
Cohérence du schéma et des flux	4
Qualité des propositions Hive/Spark	3
Clarté et justification	2

Travail en groupes de 3 à 4 étudiants. Restitution orale de 5 minutes par groupe. Note totale : 20 points.

Point d'attention

Sqoop a été retiré en 2021 et Oozie a été retiré dans les distributions récentes. Dans une proposition moderne, les étudiants peuvent citer Spark, Airflow et les formats colonnes, tout en expliquant la place historique des anciens composants.

Corrigé complet des 6 TP

Les résultats dépendant de l'environnement (identifiants de processus, capacité des disques, noms des nœuds ou identifiants YARN) peuvent varier. Le corrigé indique la démarche, les commandes et les résultats logiques attendus.